

```
In [113... import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Loading Data

```
In [114... Links=pd.read_csv('links.csv')
Movie=pd.read_csv('movies.csv')
Ratings=pd.read_csv("ratings.csv")
Tags=pd.read_csv('tags.csv')
```

```
In [115... ## Creating copy for the dataabs
Links_copy=Links.copy()
Movie_copy=Movie.copy()
Ratings_copy=Ratings.copy()
Tags_copy=Tags.copy()
```

```
In [116... Links.head()
```

```
Out[116...      movield  imdbld  tmdbld
0          1  114709    862.0
1          2  113497    8844.0
2          3  113228   15602.0
3          4  114885   31357.0
4          5  113041   11862.0
```

```
In [117... Movie.head()
```

```
Out[117...      movield      title      genres
0          1      Toy Story (1995)  Adventure|Animation|Children|Comedy|Fantasy
1          2      Jumanji (1995)      Adventure|Children|Fantasy
2          3      Grumpier Old Men (1995)      Comedy|Romance
3          4      Waiting to Exhale (1995)      Comedy|Drama|Romance
4          5  Father of the Bride Part II (1995)      Comedy
```

```
In [118... Ratings.head()
```

Out[118...

	userId	movieId	rating	timestamp
0	1	1	4.0	964982703
1	1	3	4.0	964981247
2	1	6	4.0	964982224
3	1	47	5.0	964983815
4	1	50	5.0	964982931

In [119...

Tags.head()

Out[119...

	userId	movieId	tag	timestamp
0	2	60756	funny	1445714994
1	2	60756	Highly quotable	1445714996
2	2	60756	will ferrell	1445714992
3	2	89774	Boxing story	1445715207
4	2	89774	MMA	1445715200

Merging the tables together

In [120...

Movie.merge(Ratings, 'outer', 'movieId')

Out[120...

	movieId	title	genres	userId	rating	tin
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	1.0	4.0	9.649
1	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	5.0	4.0	8.474
2	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	7.0	4.5	1.106
3	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	15.0	2.5	1.510
4	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	17.0	4.5	1.305
...	...	...	...	...	...	...
100849	193581	Black Butler: Book of the Atlantic (2017)	Action Animation Comedy Fantasy	184.0	4.0	1.537
100850	193583	No Game No Life: Zero (2017)	Animation Comedy Fantasy	184.0	3.5	1.537
100851	193585	Flint (2017)	Drama	184.0	3.5	1.537
100852	193587	Bungo Stray Dogs: Dead Apple (2018)	Action Animation	184.0	3.5	1.537
100853	193609	Andrew Dice Clay: Dice Rules (1991)	Comedy	331.0	4.0	1.537

100854 rows × 6 columns



Assigning merged table to a name

```
In [121...] Movie_ratings=Movie.merge(Ratings,'outer','movieId')
```

```
In [131...] Movie_ratings.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
Index: 100836 entries, 0 to 100853  
Data columns (total 6 columns):  
#   Column      Non-Null Count  Dtype  
---  ---  
0   movieId     100836 non-null  int64  
1   title       100836 non-null  object  
2   genres      100836 non-null  object  
3   userId      100836 non-null  float64  
4   rating      100836 non-null  float64  
5   timestamp   100836 non-null  datetime64[ns]  
dtypes: datetime64[ns](1), float64(2), int64(1), object(2)  
memory usage: 5.4+ MB
```

Checking for duplicates and missing values

```
In [123...] Movie_ratings[Movie_ratings['userId'].isna()]
```

Out[123...

	movieId	title	genres	userId	rating	timestamp
22820	1076	Innocents, The (1961)	Drama Horror Thriller	NaN	NaN	NaN
49539	2939	Niagara (1953)	Drama Thriller	NaN	NaN	NaN
53555	3338	For All Mankind (1989)	Documentary	NaN	NaN	NaN
54467	3456	Color of Paradise, The (Rang-e khoda) (1999)	Drama	NaN	NaN	NaN
60535	4194	I Know Where I'm Going! (1945)	Drama Romance War	NaN	NaN	NaN
68396	5721	Chosen, The (1981)	Drama	NaN	NaN	NaN
71896	6668	Road Home, The (Wo de fu qin mu qin) (1999)	Drama Romance	NaN	NaN	NaN
72428	6849	Scrooge (1970)	Drama Fantasy Musical	NaN	NaN	NaN
73354	7020	Proof (1991)	Comedy Drama Romance	NaN	NaN	NaN
75450	7792	Parallax View, The (1974)	Thriller	NaN	NaN	NaN
76859	8765	This Gun for Hire (1942)	Crime Film-Noir Thriller	NaN	NaN	NaN
77983	25855	Roaring Twenties, The (1939)	Crime Drama Thriller	NaN	NaN	NaN
78026	26085	Mutiny on the Bounty (1962)	Adventure Drama Romance	NaN	NaN	NaN
79080	30892	In the Realms of the Unreal (2004)	Animation Documentary	NaN	NaN	NaN
79426	32160	Twentieth Century (1934)	Comedy	NaN	NaN	NaN
79459	32371	Call Northside 777 (1948)	Crime Drama Film-Noir	NaN	NaN	NaN
80506	34482	Browning Version, The (1951)	Drama	NaN	NaN	NaN
92639	85565	Chalet Girl (2011)	Comedy Romance	NaN	NaN	NaN

There are no nans in title and genres,only in
userId,timestamp and rating that we find rows and

its 18 rows,so I will drop them respectively.

checking for duplicates

```
In [124...  ## Dropping rows in userid with nans
Movie_ratings=Movie_ratings.dropna(subset=['userId'])
```

```
In [125... Movie_ratings.isnull().sum()
Movie_ratings.duplicated().sum()
```

```
Out[125... np.int64(0)
```

changing timestamp to datetime datatype

```
In [129... Movie_ratings['timestamp']=pd.to_datetime(Movie_ratings['timestamp'])
```

```
In [130... Movie_ratings['timestamp'].head()
```

```
Out[130... 0    2000-07-30 18:45:03
1    1996-11-08 06:36:02
2    2005-01-25 06:52:26
3    2017-11-13 12:59:30
4    2011-05-18 05:28:03
Name: timestamp, dtype: datetime64[ns]
```

FEATURE ENGINEERING

1. RELEASE YEAR BY MOVIE TITLE

extracting release year from title/converting the release_year to int

```
In [132... Movie_ratings['release_year'] = Movie_ratings['title'].str.extract(r'(\d{4})')
```

```
In [133... (Movie_ratings['release_year'].unique())
#where year couldnt be found,it dropped it with 'nan'
```

```
Out[133...] array(['1995', '1994', '1996', '1976', '1992', '1967', '1993', '1964',
      '1977', '1965', '1982', '1990', '1991', '1989', '1937', '1940',
      '1969', '1981', '1973', '1970', '1955', '1959', '3000', '1968',
      '1988', '1997', '1972', '1943', '1952', '1951', '1957', '1961',
      '1958', '1954', '1934', '1944', '1960', '1963', '1942', '1941',
      '1953', '1939', '1950', '2001', '1946', '1945', '1938', '1947',
      '1935', '1936', '1956', '1949', '1932', '1975', '1974', '1971',
      '1979', '1987', '1986', '1980', '1978', '1985', '1966', '1962',
      '1983', '1984', '1948', '1933', '1931', '1922', '1600', '1998',
      '2000', '1929', '1930', '1927', '1928', '2010', '1999', '1900',
      '1926', '1919', '1921', '1925', '1923', '2002', '1776', '2003',
      '1000', '1138', '1920', '1915', '1924', '2004', '1916', '1917',
      '1492', '2046', '2005', '2006', '1902', nan, '1903', '2007',
      '1408', '2008', '2009', '2012', '2011', '2013', '2014', '2015',
      '2016', '2017', '2018', '1908', '2049', '2048'], dtype=object)
```

```
In [134...] Movie_ratings = Movie_ratings.dropna(subset=['release_year'])
# I drop the rows with nan in the release_year column
```

```
In [135...] Movie_ratings.loc[:, 'release_year'] = Movie_ratings['release_year'].astype(int)
```

```
In [140...] Movie_ratings.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 100819 entries, 0 to 100853
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   movieId         100819 non-null  int64
1   title           100819 non-null  object
2   genres          100819 non-null  object
3   userId          100819 non-null  float64
4   rating          100819 non-null  float64
5   timestamp       100819 non-null  datetime64[ns]
6   release_year    100819 non-null  int64
dtypes: datetime64[ns](1), float64(2), int64(2), object(2)
memory usage: 6.2+ MB
```

2. NUMBER OF GENRES

extracting number of genres from genres

```
In [143...] Movie_ratings.loc[:, 'num_genres'] = Movie_ratings['genres'].apply(lambda x: len(x.s
Movie_ratings.head()
```

Out[143...

	movieId	title	genres	userId	rating	timestamp
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	1.0	4.0	2000-07-30 18:45:03
1	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	5.0	4.0	1996-11-08 06:36:02
2	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	7.0	4.5	2005-01-25 06:52:26
3	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	15.0	2.5	2017-11-13 12:59:30
4	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	17.0	4.5	2011-05-18 05:28:03

3/4. RATING BY MONTH AND YEAR

extracting month and year of rating

```
In [146... Movie_ratings.loc[:, 'rating_year'] = Movie_ratings['timestamp'].dt.year
Movie_ratings.loc[:, 'rating_month'] = Movie_ratings['timestamp'].dt.month
```

```
In [153... Movie_ratings['rating_year'].head()
```

```
Out[153... 0    2000
1    1996
2    2005
3    2017
4    2011
Name: rating_year, dtype: int32
```

5.NUMBER OF RATING BY MOVIE

extracting number of rating by grouping the movieId and the rating

```
In [148... Movie_ratings.loc[:, 'num_ratings'] = Movie_ratings.groupby('movieId')['rating'].tra
```

```
In [149... Movie_ratings.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 100819 entries, 0 to 100853
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   movieId               100819 non-null  int64
1   title                 100819 non-null  object
2   genres                100819 non-null  object
3   userId                100819 non-null  float64
4   rating                100819 non-null  float64
5   timestamp             100819 non-null  datetime64[ns]
6   release_year          100819 non-null  int64
7   num_genres             100819 non-null  int64
8   rating_year           100819 non-null  int32
9   rating_month          100819 non-null  int32
10  num_ratings            100819 non-null  int64
dtypes: datetime64[ns](1), float64(2), int32(2), int64(4), object(2)
memory usage: 8.5+ MB
```

```
In [226... subset1=Movie_ratings[['movieId', 'num_ratings']]
subset1
```

```
Out[226...
   movieId  num_ratings
0         1          215
1         1          215
2         1          215
3         1          215
4         1          215
...      ...          ...
100849    193581          1
100850    193583          1
100851    193585          1
100852    193587          1
100853    193609          1
```

100819 rows × 2 columns

```
In [150... Movie_ratings.head()
```

Out[150...

	movieId	title	genres	userId	rating	timestamp
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	1.0	4.0	2000-07-30 18:45:03
1	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	5.0	4.0	1996-11-08 06:36:02
2	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	7.0	4.5	2005-01-25 06:52:26
3	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	15.0	2.5	2017-11-13 12:59:30
4	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	17.0	4.5	2011-05-18 05:28:03



In []:

6. MOVIE AGE AS AT TIME OF RATING

extracting the age of the movie as at the time rating was given, by subtracting rating year from the release year

```
In [155... Movie_ratings.loc[:, 'movie_age_at_rating'] = Movie_ratings['rating_year'] - Movie_r
```

```
In [215... Movie_ratings['movie_age_at_rating']
```

Out[215...

```
0      5
1      1
2     10
3     22
4     16
..
100849  1
100850  1
100851  1
100852  0
100853  27
Name: movie_age_at_rating, Length: 100819, dtype: int64
```

```
In [214... subset3=Movie_ratings[['movieId', 'title', 'movie_age_at_rating' ]]
subset3
```

Out[214...

	movieId	title	movie_age_at_rating
0	1	Toy Story (1995)	5
1	1	Toy Story (1995)	1
2	1	Toy Story (1995)	10
3	1	Toy Story (1995)	22
4	1	Toy Story (1995)	16
...	...	...	...
100849	193581	Black Butler: Book of the Atlantic (2017)	1
100850	193583	No Game No Life: Zero (2017)	1
100851	193585	Flint (2017)	1
100852	193587	Bungo Stray Dogs: Dead Apple (2018)	0
100853	193609	Andrew Dice Clay: Dice Rules (1991)	27

100819 rows × 3 columns

MOST COMMON TAG

In [194...

```
Movie_ratings_tag=Movie_ratings.merge(Tags,on=['userId','movieId'],how='inner')
```

merging already existed table with tag table to be able to get'top 10 most common movie tag'

In [195...

```
Movie_ratings_tag.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3473 entries, 0 to 3472
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   movieId               3473 non-null   int64
1   title                 3473 non-null   object
2   genres                3473 non-null   object
3   userId               3473 non-null   float64
4   rating                3473 non-null   float64
5   timestamp_x           3473 non-null   datetime64[ns]
6   release_year          3473 non-null   int64
7   num_genres            3473 non-null   int64
8   rating_year           3473 non-null   int32
9   rating_month          3473 non-null   int32
10  num_ratings           3473 non-null   int64
11  movie_age_at_rating    3473 non-null   int64
12  tag                   3473 non-null   object
13  timestamp_y           3473 non-null   int64
dtypes: datetime64[ns](1), float64(2), int32(2), int64(6), object(3)
memory usage: 352.9+ KB
```

```
In [197... Movie_ratings_tag.drop_duplicates()
Movie_ratings_tag.head()
```

```
Out[197...
   movieId  title  genres  userId  rating  timestamp_
0         1  Toy Story (1995)  Adventure|Animation|Children|Comedy|Fantasy  336.0    4.0  2005-07-21 17:48:4
1         1  Toy Story (1995)  Adventure|Animation|Children|Comedy|Fantasy  474.0    4.0  2001-01-01 02:36:0
2         1  Toy Story (1995)  Adventure|Animation|Children|Comedy|Fantasy  567.0    3.5  2018-05-01 18:33:2
3         2  Jumanji (1995)      Adventure|Children|Fantasy      62.0    4.0  2018-06-11 22:51:3
4         2  Jumanji (1995)      Adventure|Children|Fantasy      62.0    4.0  2018-06-11 22:51:3
```

```
In [200... Movie_ratings_tag=Movie_ratings_tag.drop(columns=['timestamp_y'])
Movie_ratings_tag=Movie_ratings_tag.rename(columns={'timestamp_x': 'timestamp'})
```

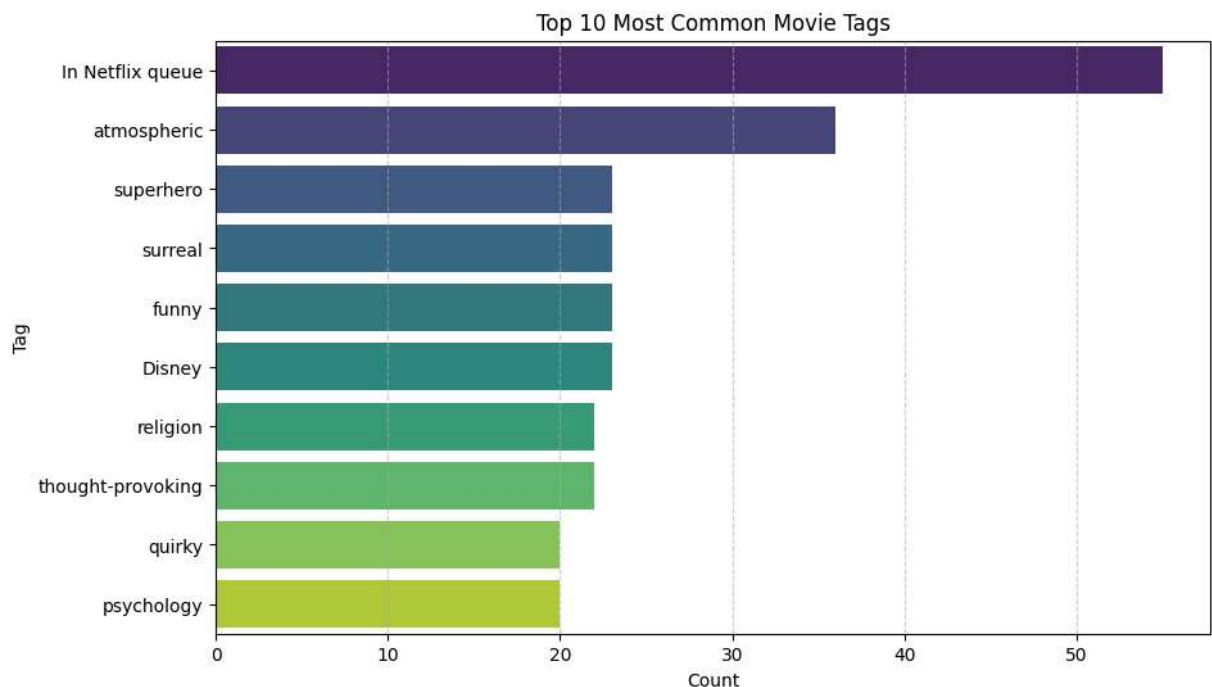
```
In [229... Movie_ratings.to_csv(r"C:\Users\ROSZ\Desktop\Data Analysisr\BLINKIT/MovieLensData.csv")
```

EXPLORATORY DATA ANALYSIS

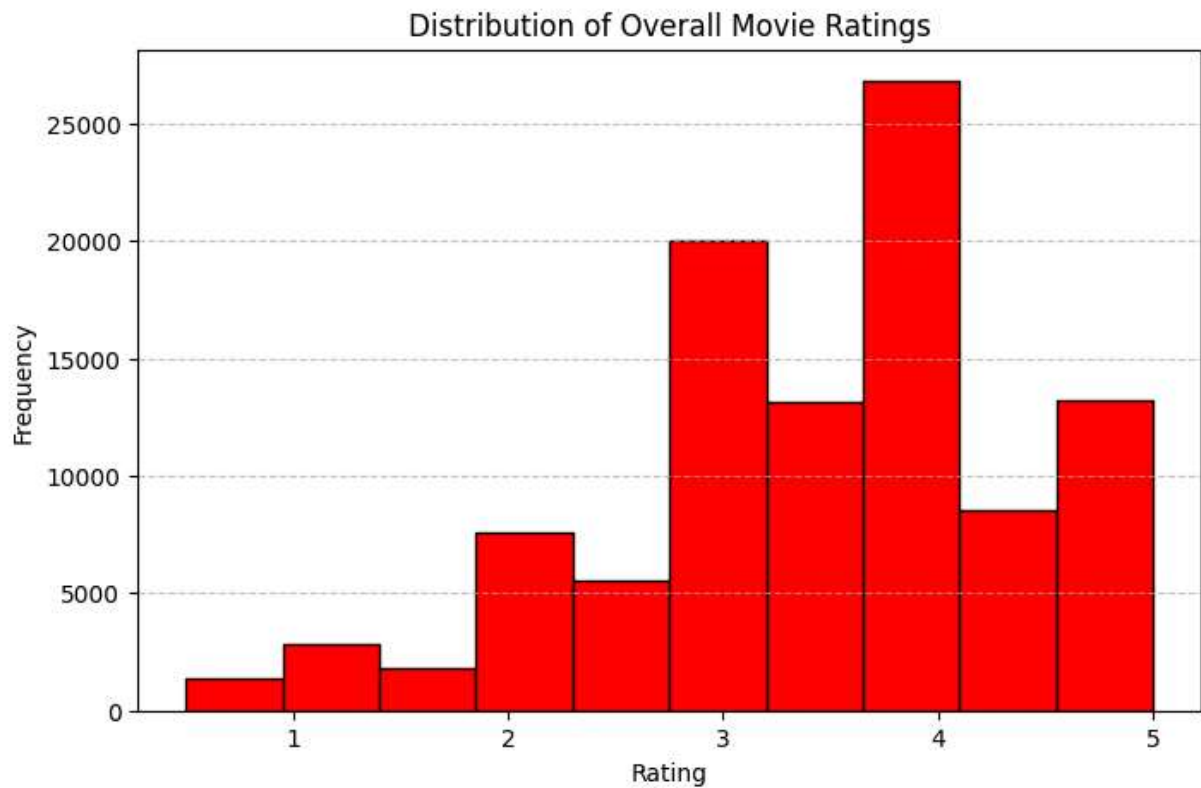
```
In [205... # Count most common tags and reset index
tag_counts = Movie_ratings_tag['tag'].value_counts().reset_index()
tag_counts.columns = ['tag', 'count']

top_tags = tag_counts.head(10)

plt.figure(figsize=(10,6))
sns.barplot(x='count', y='tag', data=top_tags, palette='viridis')
plt.title('Top 10 Most Common Movie Tags')
plt.xlabel('Count')
plt.ylabel('Tag')
plt.grid(axis='x', linestyle='--', alpha=0.6)
plt.show()
```



```
In [162... plt.figure(figsize=(8,5))
plt.hist(Movie_ratings['rating'], bins=10, color='red', edgecolor='black')
plt.title('Distribution of Overall Movie Ratings')
plt.xlabel('Rating')
plt.ylabel('Frequency')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



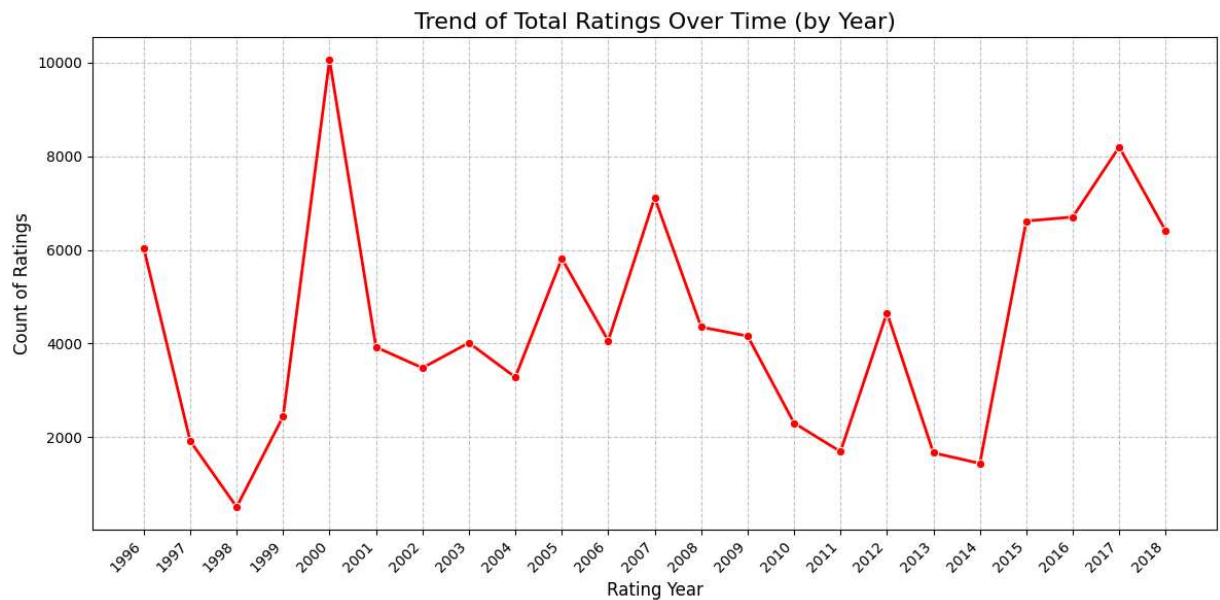
```
In [180... ratings_per_year = Movie_ratings['rating_year'].value_counts().sort_index()

ratings_df = ratings_per_year.reset_index()
ratings_df.columns = ['RatingYear', 'Count']

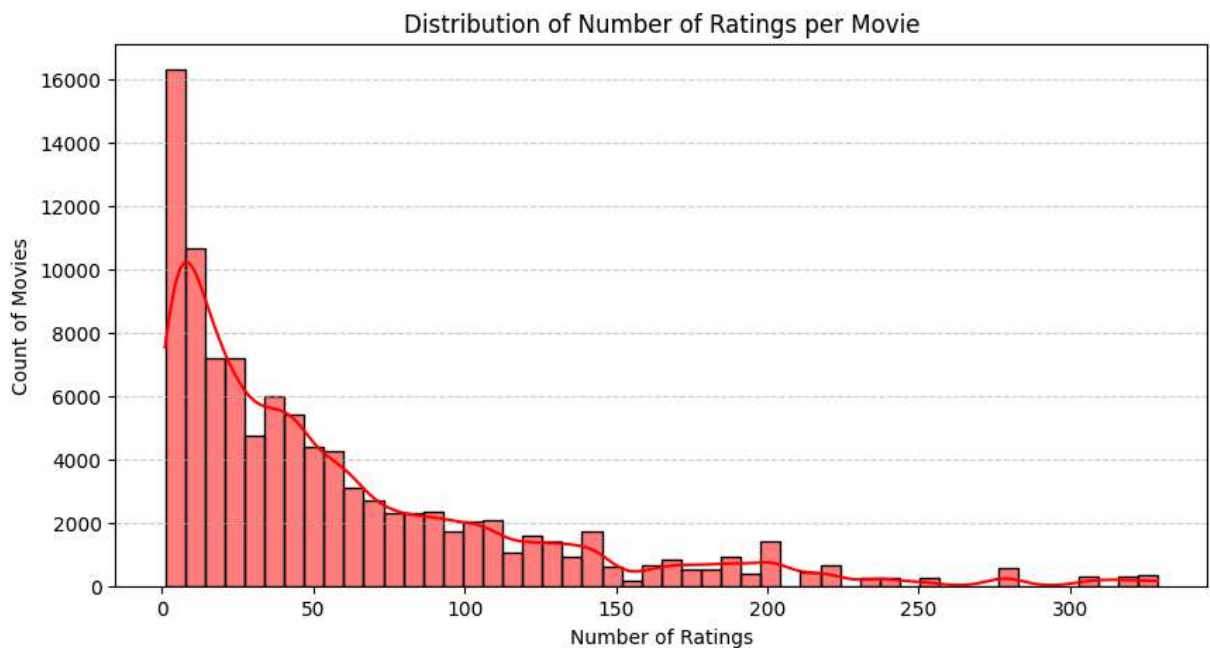
plt.figure(figsize=(12, 6))
sns.lineplot(
    x='RatingYear',
    y='Count',
    data=ratings_df,
    marker='o',
    color='red',
    linewidth=2
)

plt.title('Trend of Total Ratings Over Time (by Year)', fontsize=16)
plt.xlabel('Rating Year', fontsize=12)
plt.ylabel('Count of Ratings', fontsize=12)
plt.xticks(ratings_df['RatingYear'].unique(), rotation=45, ha='right')

plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.savefig('ratings_trend_by_year.png')
plt.show()
```



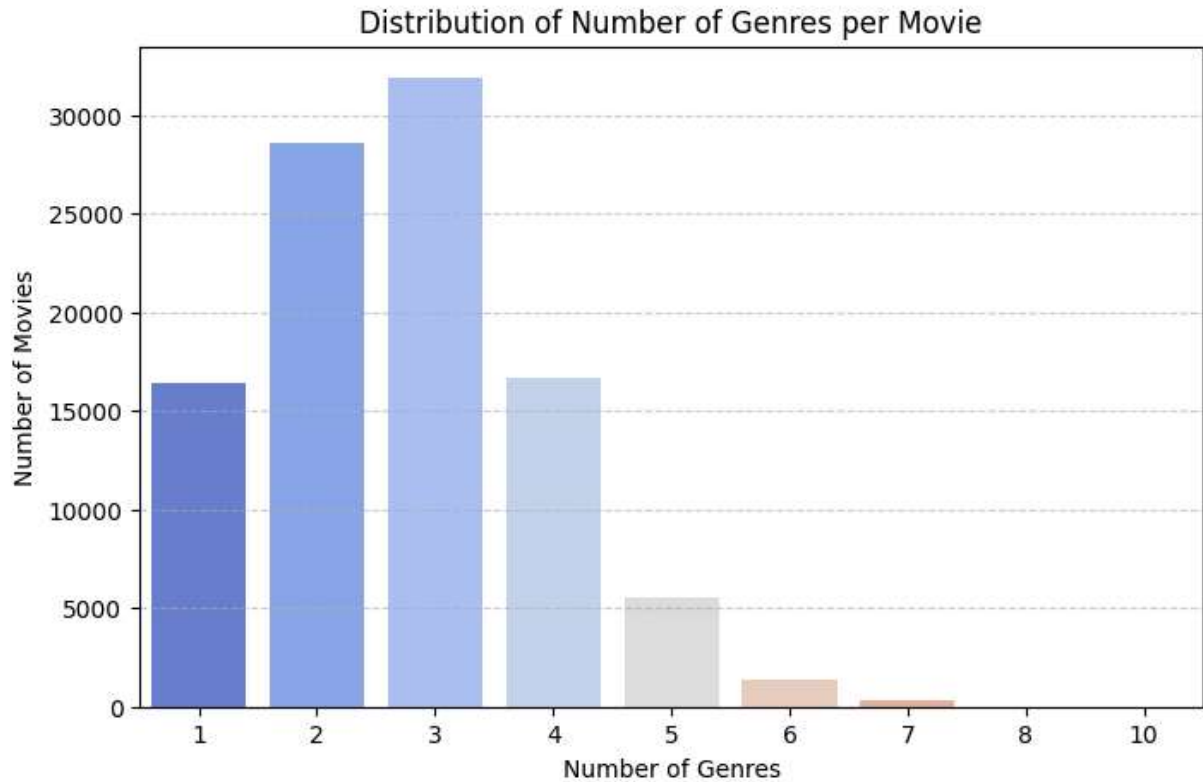
```
In [184... plt.figure(figsize=(10,5))
sns.histplot(Movie_ratings['num_ratings'], bins=50, kde=True, color='red')
plt.title('Distribution of Number of Ratings per Movie')
plt.xlabel('Number of Ratings')
plt.ylabel('Count of Movies')
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.show()
```



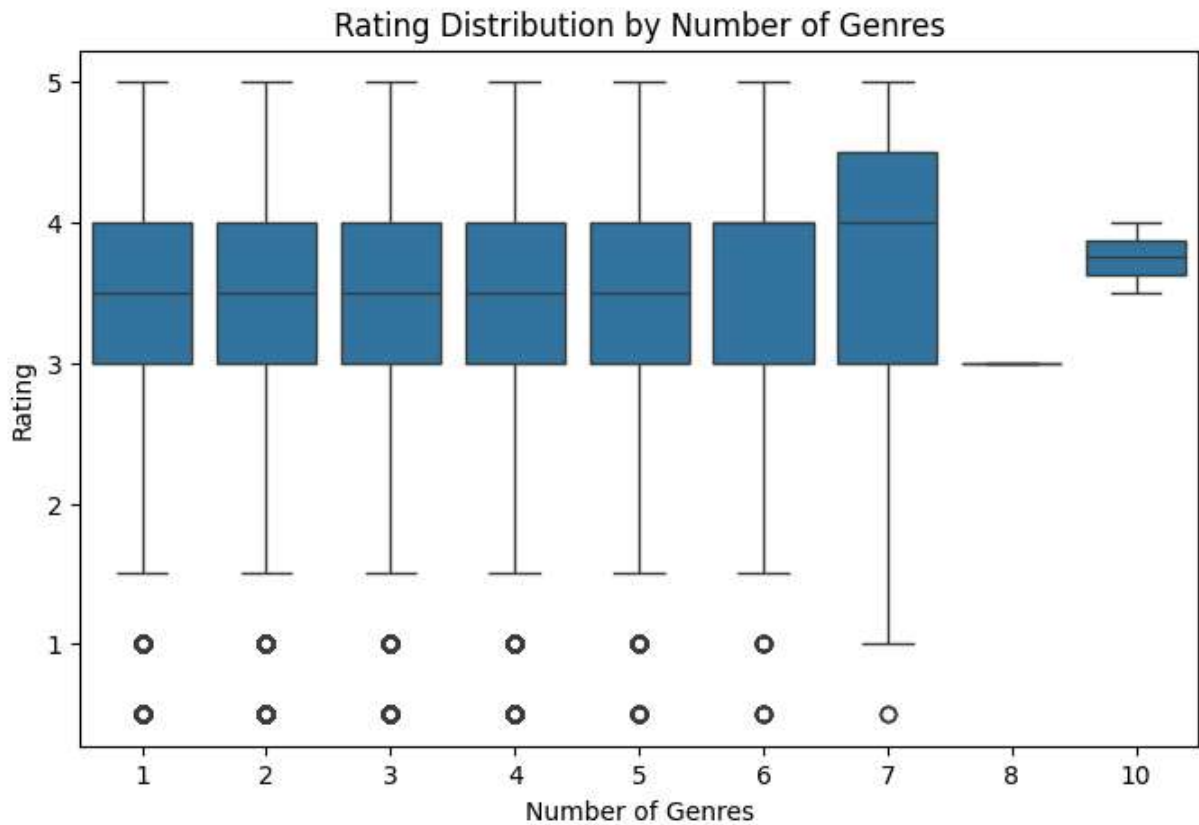
```
In [191... import warnings
warnings.filterwarnings('ignore')

plt.figure(figsize=(8,5))
sns.countplot(x='num_genres', data=Movie_ratings, palette='coolwarm', legend=False)
plt.title('Distribution of Number of Genres per Movie')
plt.xlabel('Number of Genres')
plt.ylabel('Number of Movies')
```

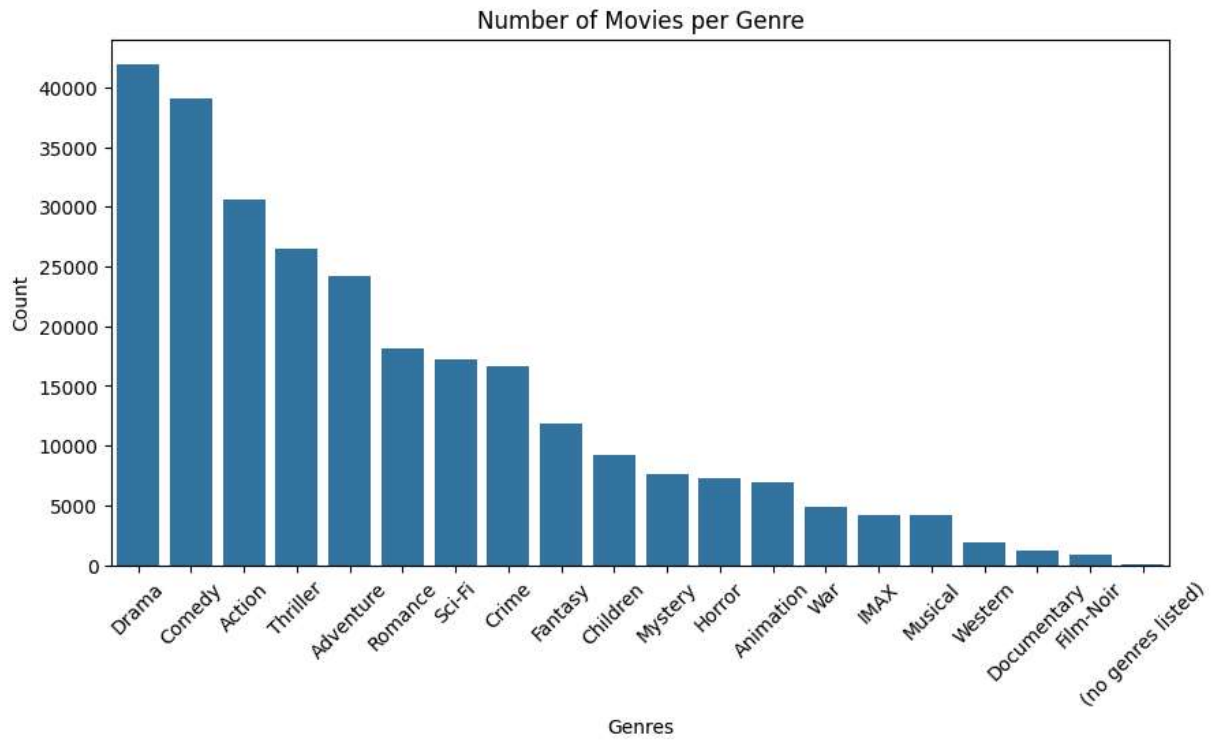
```
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.show()
```



```
In [218... plt.figure(figsize=(8,5))
sns.boxplot(x='num_genres', y='rating', data=Movie_ratings)
plt.title('Rating Distribution by Number of Genres')
plt.xlabel('Number of Genres')
plt.ylabel('Rating')
plt.show()
```



```
In [228... genre_counts = Movie_ratings['genres'].str.split('|', expand=True).stack().value_counts()
plt.figure(figsize=(10,5))
sns.barplot(x=genre_counts.index, y=genre_counts.values)
plt.title('Number of Movies per Genre')
plt.xticks(rotation=45)
plt.ylabel('Count')
plt.xlabel('Genres')
plt.show()
```



In []: